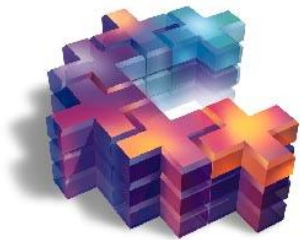


Compilation Techniques

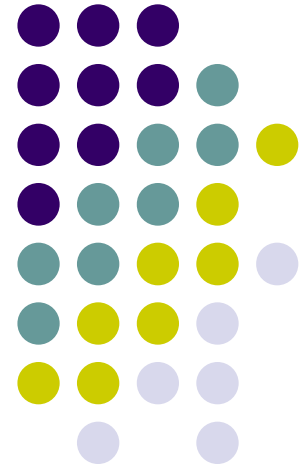
Ch5: SYNTAX Analysis



By Dr. Ameni Mejri



Ecole Supérieure Privée
d'Ingénierie et de Technologies

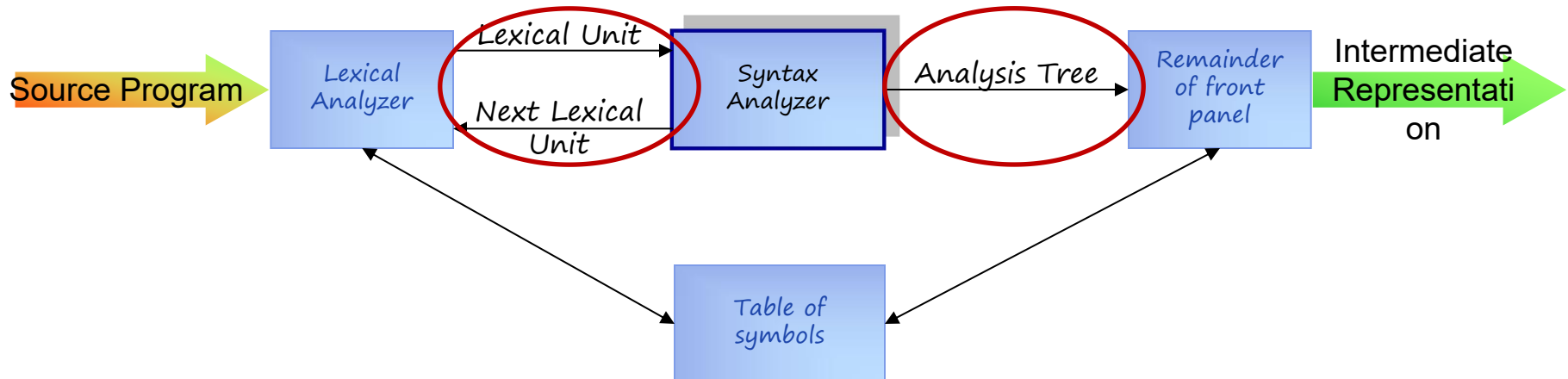


Introduction

- Purpose of syntactic analysis: to transform a stream of lexical units into an abstract tree.
- Programming language: rules prescribing the syntactic structure of well-formed programs.
- C language, Program = set of Functions.
 - Function: made up of blocks.
 - A block: made up of instructions.
 - An instruction: made up of expressions.
 - An expression: made up of lexical units.
 - Etc.
- The syntax of a language can be described by context free grammars (BNF notation - Backus-Naur Form).
- Grammar: precise, easy-to-understand syntactic specification of a programming language.

Role of the syntax analyzer

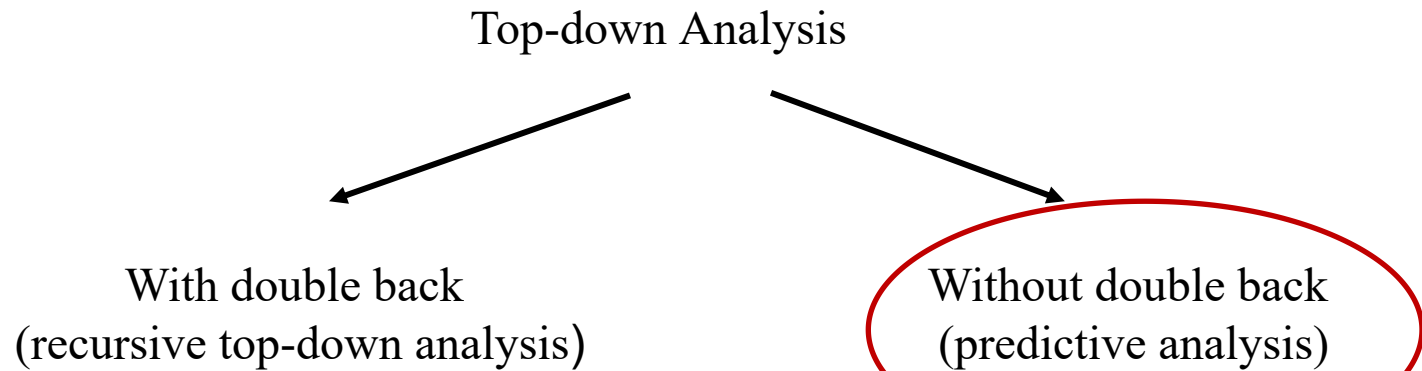
- The syntactic analyzer obtains a string of lexical units from the lexical analyzer.
- It checks that the string can be generated by the grammar of the source language.
- It reports every syntax error in an intelligible way.
- It supports the most common errors so that it can continue analyzing the remaining code.



Role of the syntax analyzer

- Two types of analysis are retained for compilers: Ascending (Bottom-up) or Descending (Top-down).
- Descending analyzers build analysis trees from top (ROOT) to bottom (LEAF).
- Ascending analyzers start from the leaves and work their way back to the root.
- In both cases, the parsing input is conventionally explored from left to right, one symbol at a time.
- The error handler must :
 - Indicate the presence of errors clearly and precisely.
 - Process each error so that subsequent errors can be detected.
- The error handler must not penalize the execution of correct programs.

Top-down SYNTAXIC analysis



1. Unambiguous
2. Left factorized
3. Left non-recursive

Left factorizATIOn

Left factorization algorithm

Inputs : A G grammar without cycles and empty productions.

OutPuts : A left factorized grammar equivalent to G.

For each non-terminal A , find the longest prefix α to at least two of its alternatives.

If $\alpha \neq \varepsilon$, then replace all A-productions:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$

γ Represents all alternatives that do not begin with α with :

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

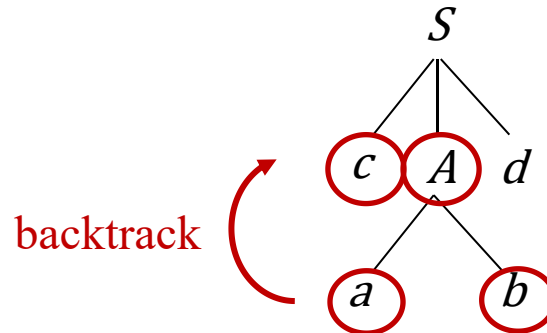
Repeat until none of the alternatives for the same non-terminal have a common prefix.

Example

$\{S \rightarrow cAd$
 $\{A \rightarrow ab \mid a$
 $w = cad$

$c \underline{a} d$
 $\uparrow \uparrow \uparrow$

Go back



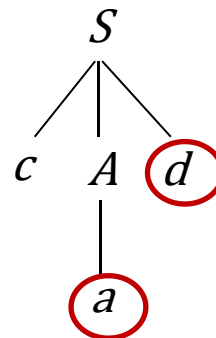
Failure
return to previous choice

$S \rightarrow c \underline{A} d$

$S \rightarrow c \underline{A} d \text{X} \rightarrow c a b d$

$S \rightarrow c \underline{A} d \rightarrow c a d$

Success



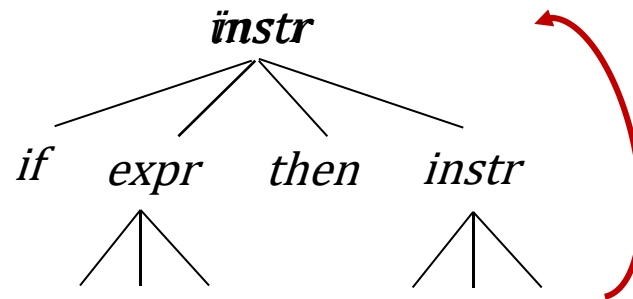
In this example, we are dealing with a general form of Top-down Syntactic analysis where backtracking is possible with repeated passages over the source text.

In practice, backtracking is rarely necessary, and grammars can be adapted to perform top-down Top-down Syntactic analysis without backtracking.

Example

$$\left\{ \begin{array}{l} instr \rightarrow if\ expr\ then\ instr \\ \quad | if\ expr\ then\ instr\ else\ instr \\ \quad | other \end{array} \right.$$

Which alternative to choose after reading the if ?



backtrack

$w = if\ (speed \geq 110)\ then\ amende = true\ else\ amende = false$

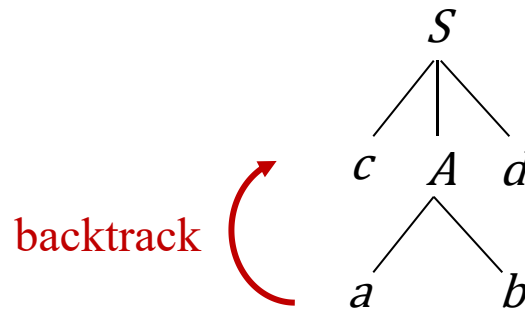


Go back

Example

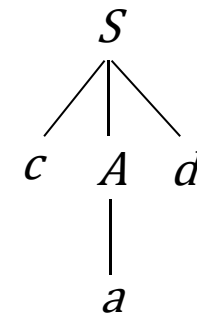
What's causing the problem?

$\{ S \rightarrow cAd$
 $\{ A \rightarrow \underline{ab} \mid \underline{a}$
 $w = \underline{c} \underline{a} \underline{d}$



failure
return to previous choice

$\{ instr \rightarrow \underline{if\ expr\ then\ instr}$
 $\quad \quad \quad \underline{| if\ expr\ then\ instr\ else\ instr}$
 $\quad \quad \quad | other$



$S \rightarrow c \underline{A} d$

$S \rightarrow c \underline{A} d \text{X} c a b d$

$S \rightarrow c \underline{A} d \rightarrow c a d$

➡ The problem is factoring to the left

Propose an equivalent grammar that denotes the same language while eliminating the problem?

Left factorization

example

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow \underline{a}b \mid \underline{a} \end{cases}$$

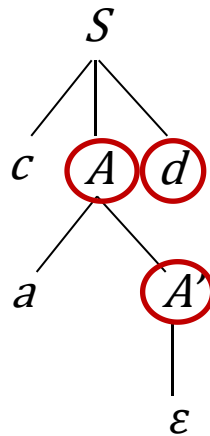
Left factorization



$$\begin{cases} S \rightarrow cAd \\ A \rightarrow a(b \mid \varepsilon) \end{cases}$$

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow b \mid \varepsilon \end{cases}$$

$w = cad$



Success

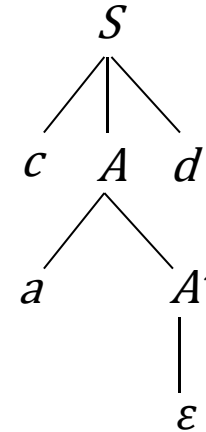
$$S \rightarrow c \underline{A} d \rightarrow c a \underline{A'} d \rightarrow c a d$$

Propose an equivalent grammar that denotes the same language while eliminating the problem?

Left factorization

example

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow \underline{ab} \mid \underline{a} \end{cases} \xrightarrow{\text{Left factorization}} \begin{cases} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow \underline{b} \mid \varepsilon \end{cases}$$



To generalize: if

$$S \rightarrow \underline{\alpha}\beta_1\underline{\alpha}\beta_2\cdots\underline{\alpha}\beta_n\gamma_1\cdots\gamma_m \xrightarrow[\text{Then}]{\text{Left factorization}} \begin{cases} S \rightarrow \underline{\alpha}S' \mid \gamma_1\cdots\gamma_m \\ S' \rightarrow \underline{\beta_1}\beta_2\cdots\beta_n \end{cases}$$

$$\begin{cases} S \rightarrow aRbc \mid abc \\ R \rightarrow aRTb \mid aTb \\ T \rightarrow bb \mid cc \end{cases} \xrightarrow{\text{Left factorization}} \begin{cases} S \rightarrow aS' \\ S' \rightarrow Rbc \mid bc \\ R \rightarrow aR' \\ R' \rightarrow RTb \mid Tb \\ T \rightarrow bb \mid cc \end{cases}$$

left and right recursion

- A grammar is recursive if it contains a non-terminal A that derives into a production containing A .
- A production of a grammar is left-recursive if the leftmost symbol of the right part of the production is the same as the non-terminal of the left part of the production (there is a derivation $A \Rightarrow A\alpha$ (α any string)).

Example : $A \rightarrow A\alpha$

- A production of a grammar is right-recursive if the rightmost symbol in the right part of the production is the same as the non-terminal in the left part of the production.

Example : $A \rightarrow \alpha A$

α is a sequence of terminals and non-terminals not containing A .

How to Remove The left-hand recursion

- Top-down parsing methods do not work with left-recursive grammars.
- The recursive descent parser can loop indefinitely.
- For descending parsers, left recursion must be eliminated.
- Rewriting rules to eliminate left recursion.
- A is non-terminal in the following two productions :

$$A \rightarrow A\alpha \mid \beta$$

α et β are sequences of terminals and non-terminals not containing A .

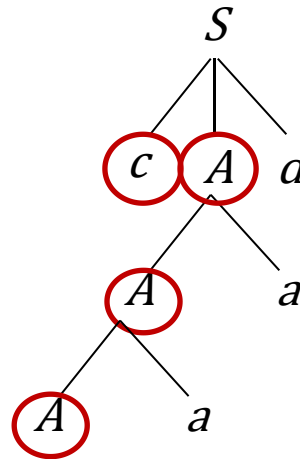
Example : $expression \rightarrow expression + term \mid term$

left-hand recursion

Example

$\{S \rightarrow cAd$
 $\{A \rightarrow \underline{A}a\} a$
 $w = cad$

$\underline{c} \underline{a} d$
↑ ↑



Endless loop

$S \rightarrow c \underline{A} d$

$S \rightarrow c \underline{A} d \rightarrow c A a d$

$S \rightarrow c \underline{A} d \rightarrow c A a d \rightarrow c A a a d$

...

The problem is that grammar is left recursive

Sometimes you can't even turn back to perform a backtracking !!

left-hand recursion

Example

$$\underline{A} \rightarrow \underline{A\alpha} \mid \underline{\beta}$$

Suggest some language words?

$$\beta$$

$$\beta\alpha\cdots\alpha$$

$$\alpha$$

$$\alpha\cdots\alpha$$

$$A \rightarrow A\alpha \rightarrow A\alpha\alpha \rightarrow A\alpha\alpha\alpha \rightarrow \beta\alpha\alpha\alpha$$

Suggest a grammar that denotes the same language without being **Left-recursive** ?

$$\begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \varepsilon \end{cases}$$

If we generalize:

After eliminating recursion

$$\begin{cases} \underline{A} \rightarrow \underline{A\alpha_1} \mid \underline{A\alpha_2} \mid \cdots \mid \underline{A\alpha_n} \mid \underline{\beta_1} \mid \cdots \mid \underline{\beta_m} \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_n A' \mid \varepsilon \end{cases}$$

left-hand recursion

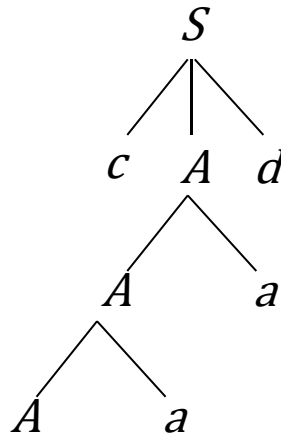
Example

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow \underline{A}a \mid a \end{cases}$$

$\underline{w} = \underline{cad}$

$c a d$
↑↑↑

Endless Loop



$$S \rightarrow c \underline{A} d$$

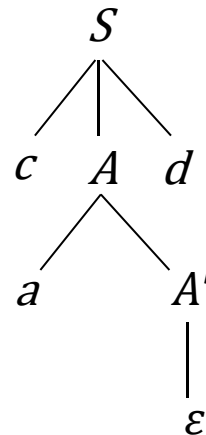
$$S \rightarrow c \underline{A} d \rightarrow c A a d$$

$$S \rightarrow c \underline{A} d \rightarrow c A a d \rightarrow c A a a d$$

...

After eliminating recursion :

$$\begin{cases} S \rightarrow cAd \\ A \rightarrow aA' \\ A' \rightarrow aA' \mid \varepsilon \end{cases}$$



Success

left-hand recursion

Example

If we generalize:

After eliminating recursion

$$\{\underline{A} \rightarrow \underline{A}\alpha_1 \mid \underline{A}\alpha_2 \mid \cdots \mid \underline{A}\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m\} \longrightarrow \begin{cases} A \rightarrow \beta_1 A' \mid \cdots \mid \beta_m A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_n A' \mid \varepsilon \end{cases}$$

After eliminating recursion

$$\begin{cases} \underline{S} \rightarrow aS \mid \underline{S}b \mid c \\ aSb \rightarrow \underline{S}a \mid bS \end{cases} \longrightarrow \begin{cases} S \rightarrow aSS' \mid cS' \\ S' \rightarrow bS' \mid \varepsilon \\ aSb \rightarrow Sa \mid bS \end{cases}$$

left-hand recursion

Example

If we generalize:

After eliminating recursion

$$\{\underline{A} \rightarrow \underline{A}\alpha_1 \mid \underline{A}\alpha_2 \mid \cdots \mid \underline{A}\alpha_n \mid \beta_1 \mid \cdots \mid \beta_m\} \longrightarrow \begin{cases} A \rightarrow \beta_1 A' \mid \cdots \mid \beta_m A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_n A' \mid \varepsilon \end{cases}$$

After eliminating recursion

$$\begin{cases} E \rightarrow \overset{\alpha}{E} + T \mid T \\ \underline{T} \rightarrow \underline{T} * F \mid F \\ \underline{F} \rightarrow \underline{(E)} \mid id \end{cases} \longrightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow \overline{+} TE' \mid \varepsilon \\ T \rightarrow \overline{T} * F \mid F \\ F \rightarrow (E) \mid id \end{cases} \longrightarrow \begin{cases} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \varepsilon \\ F \rightarrow (E) \mid id \end{cases}$$

Suppressing hidden left-hand recursion

Rules for eliminating immediate left-hand recursion

- Group A -productions as follows :

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ No β_i begins with an A

- A -productions are replaced by :

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$

- This algorithm eliminates all immediate left-hand recursions.
- It does not eliminate left-hand recursions involving derivations containing at least two steps.

$S \rightarrow Aa \mid B$

$A \rightarrow Ac \mid Sd \mid \varepsilon$

The non-terminal S is left-recursive (but not immediately) because

$S \Rightarrow Aa \Rightarrow Sda$

Suppressing hidden left-hand recursion

Left recursion elimination algorithm

InPuts : A G grammar without cycles and empty productions

Outputs : A grammar without left-hand recursion equivalent to G

BEGIN

Order non-terminals $A_1, A_2, \dots, A_n$;

FOR $i \leftarrow 1$ **TO** n **DO**

FOR $j \leftarrow 1$ **TO** $i - 1$ **DO**

replace each production of the form $A_i \rightarrow A_j \gamma$ by the productions
 $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$ ($A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are current productions)

END FOR

eliminate immediate recursivity of A_i -productions

END FOR

END

Suppressing hidden left-hand recursion

Example :

$S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$

- Non-terminals are classified in the following order S, A
- There is no immediate left-hand recursion in S -productions. Nothing happens during the execution of step 2 for $i = 1$.
- For $i = 2$, we substitute the S -productions in $A \rightarrow Sd$ to obtain the following A -productions:

$A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$

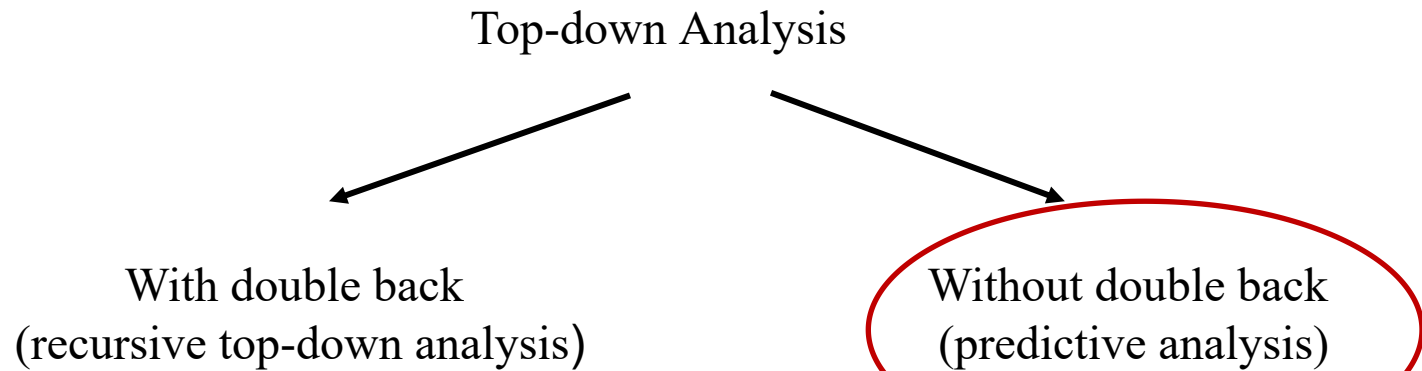
- By eliminating immediate left-hand recursions, we obtain the following grammar:

$S \rightarrow Aa \mid b$

$A \rightarrow bdA' \mid A'$

$A' \rightarrow cA' \mid adA' \mid \varepsilon$

Top-down SYNTAXIC analysis



1. Unambiguous
2. Left factorized
3. Left non-recursive

Top-down SYNTAXIC analysis

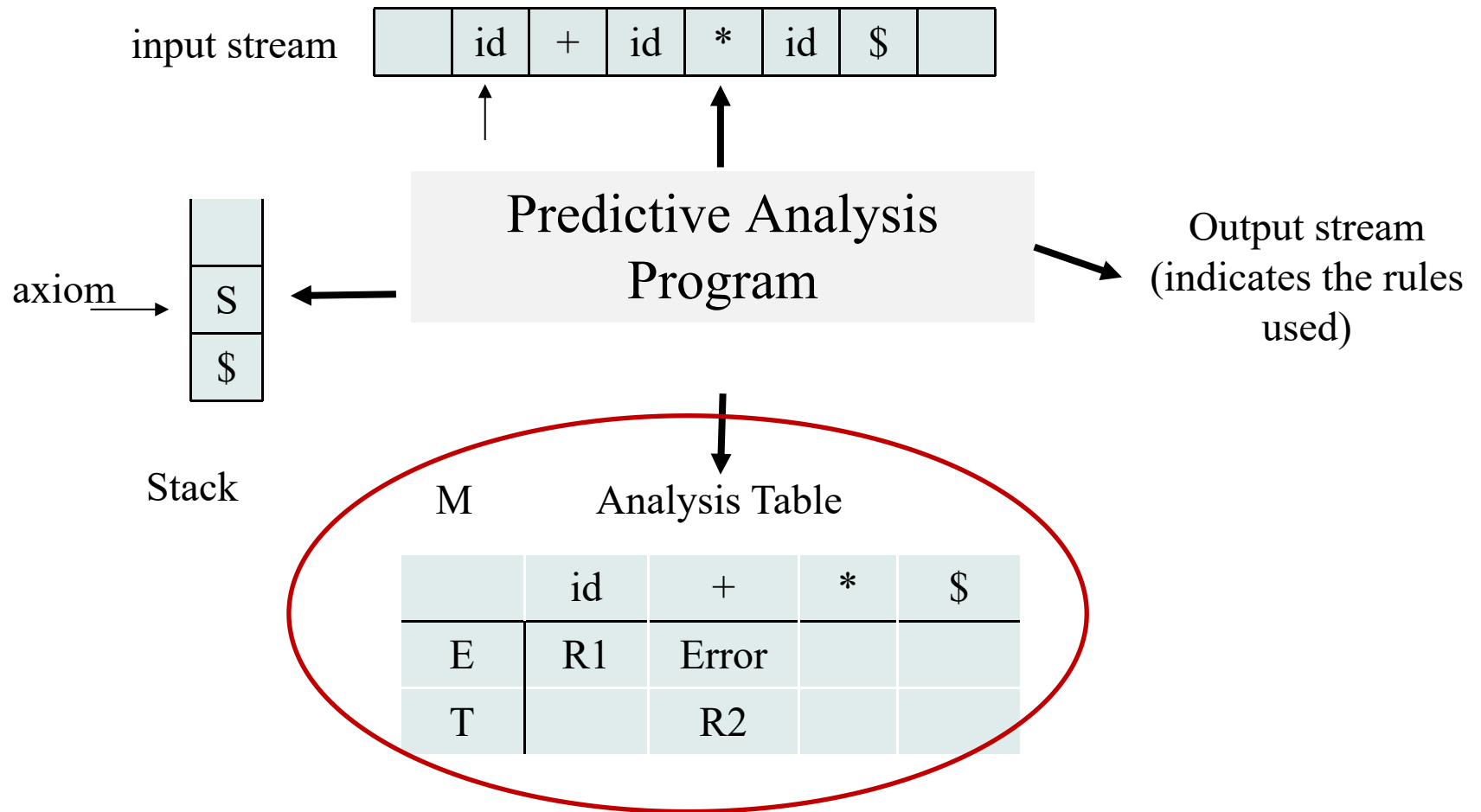
Predictive analyzers

- Given a symbol a as input and a non-terminal A to be expanded, determine whether an alternative of the production $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ is the only alternative that can derive into a string starting with a .

```
instructionn → if expression then instruction else  
              | while expression do instruction  
              | begin instructions_list end
```

In this case, the **if**, **while** and **begin** keywords tell us which alternative to take to develop the instruction.

Predictive analysis



Calculating the first

Let G be the following grammar :

$$S \rightarrow aA \mid bB$$

$$A \rightarrow Bc \mid BC$$

$$B \rightarrow bc \mid a$$

$$C \rightarrow c \mid \varepsilon$$

$$W = a\cdots?$$

$$W = b\cdots?$$

$$W = c\cdots?$$

$$S \overset{\text{X}}{\rightarrow}^* c\cdots$$

The first?

$$\text{Fr}(S) = \{a, b\}$$

$S \rightarrow^*$ the first terminal ...

$$\text{Fr}(A) = \text{Fr}(B) = \{b, a\}$$

$$S \rightarrow \textcircled{a} A \rightarrow aBc\cdots$$

$$\text{Fr}(B) = \{b, a\}$$

$$\text{F}(C) = \{c, \varepsilon\}$$

Calculating the first

Let G be the following grammar :

$$S \rightarrow aA \mid bB$$

$$A \rightarrow Bc \mid BC$$

$$B \rightarrow bc \mid a \mid \varepsilon$$

$$C \rightarrow c \mid \varepsilon$$

$$\text{Fr}(S) = \{a, b\}$$

$$A \rightarrow BC \rightarrow \varepsilon C \rightarrow \varepsilon$$

$$\text{Fr}(A) = \text{Fr}(B) \setminus \{\varepsilon\} \cup \{c\} \cup \text{Fr}(C) = \{b, a, c, \varepsilon\}$$

$$\text{Fr}(B) = \{b, a\} \quad \text{Fr}(B) = \{b, a, \varepsilon\}$$

$$\text{Fr}(C) = \{c, \varepsilon\}$$

Calculating the first

Let G be the following grammar :

$$S \rightarrow aA \mid bB$$

$$A \rightarrow Bc \mid BCe$$

$$B \rightarrow bc \mid a \mid \epsilon$$

$$C \rightarrow c \mid \epsilon$$

$$\text{Fr}(S) = \{a, b\}$$

$$\text{Fr}(A) = \text{Fr}(B) \setminus \{\epsilon\} \cup \{c\} \cup \text{Fr}(C) \setminus \{\epsilon\} \cup \{e\} = \{b, a, c, e\}$$

$$\text{Fr}(B) = \{b, a\} \quad \text{Fr}(B) = \{b, a, \epsilon\}$$

$$\text{Fr}(C) = \{c, \epsilon\}$$

Example of application

Consider the following grammar G_1 :

$$\begin{array}{ll} S \rightarrow aSb \mid cd \mid bSAe & \text{Fr}(S) = \{a, c, b\} \\ A \rightarrow aAdB \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} \\ B \rightarrow bb & \text{Fr}(B) = \{b\} \end{array}$$

Consider the following grammar G_2 :

$$\begin{array}{ll} S \rightarrow ABCe & \text{Fr}(S) = \text{Fr}(A) \setminus \{\varepsilon\} \cup \text{Fr}(B) \setminus \{\varepsilon\} \cup \text{Fr}(C) = \{a, b, c, d\} \\ A \rightarrow aA \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} \\ B \rightarrow bB \mid cB \mid \varepsilon & \text{Fr}(B) = \{b, c, \varepsilon\} \\ C \rightarrow de \mid da \mid dA & \text{Fr}(C) = \{d\} \end{array}$$

Example of application

Consider the following grammar G_1 :

$$\begin{array}{ll} S \rightarrow aSb \mid cd \mid bSAe & \text{Fr}(S) = \{a, c, b\} \\ A \rightarrow aAdB \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} \\ B \rightarrow bb & \text{Fr}(B) = \{b\} \end{array}$$

Consider the following grammar G_2 :

$$\begin{array}{ll} S \rightarrow ABCe & \text{Fr}(S) = \text{Fr}(A) \setminus \{\varepsilon\} \cup \text{Fr}(B) \setminus \{\varepsilon\} \cup \text{Fr}(C) \setminus \{\varepsilon\} \cup \{e\} = \{a, b, c, d, e\} \\ A \rightarrow aA \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} \\ B \rightarrow bB \mid cB \mid \varepsilon & \text{Fr}(B) = \{b, c, \varepsilon\} \\ C \rightarrow de \mid da \mid dA \mid \varepsilon & \text{Fr}(C) = \{d, \varepsilon\} \end{array}$$

Calculating the next

Consider the following grammar G :

S	$\rightarrow$	$Aa \mid \underline{BSb}$
A	$\rightarrow$	$\underline{Bc} \mid \underline{BC}$
B	$\rightarrow$	$bc \mid a$
C	$\rightarrow$	$c \mid \varepsilon$

The next?

$S \rightarrow^* \dots A \underline{a} \dots$

The first

$S \rightarrow Aa \rightarrow B \underline{c} a \dots$

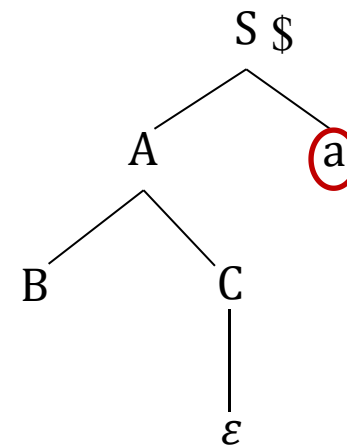
$S \rightarrow Aa \rightarrow B \underline{C} a \dots$

$\text{Next}(S) = \{b, \$\}$

$\text{Next}(A) = \{a\}$

$\text{Next}(B) = \text{Fr}(S) \cup \{c\} \cup \text{Fr}(C) \setminus \{\varepsilon\} \cup \text{Next}(A)$

$\text{Next}(C) = \text{Next}(A) = \{a\}$



Calculating the next

Consider the following grammar G :

S	$\rightarrow$	$Aa \mid BSb$
A	$\rightarrow$	$\underline{Bc} \mid \underline{BC}d$
B	$\rightarrow$	$bc \mid a$
C	$\rightarrow$	$c \mid \varepsilon$

The next?

$S \rightarrow^* \dots Aa \dots$

The first

$S \rightarrow Aa \rightarrow Bc a \dots$

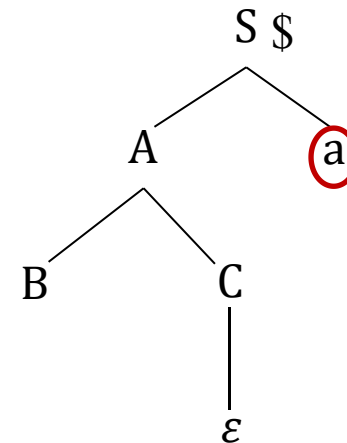
$S \rightarrow Aa \rightarrow BC a \dots$

$\text{Next}(S) = \{b, \$\}$

$\text{Next}(A) = \{a\}$

$\text{Next}(B) = \text{Fr}(S) \cup \{c\} \cup \text{Fr}(C) \setminus \{\varepsilon\} \cup \{d\}$

$\text{Next}(C) = \text{Next}(A) = \{a\}$



Calculating the next

Consider the following grammar G :

S	$\rightarrow$	$Aa \mid BSb$
A	$\rightarrow$	$\underline{Bc} \mid \underline{BCD}$
B	$\rightarrow$	$bc \mid a$
C	$\rightarrow$	$c \mid \varepsilon$

The next?

$S \rightarrow^* \dots Aa \dots$

The first

$S \rightarrow Aa \rightarrow Bc a \dots$

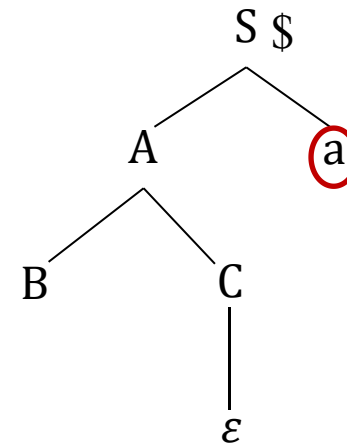
$S \rightarrow Aa \rightarrow BCa \dots$

$\text{Next}(S) = \{b, \quad\}$

$\text{Next}(A) = \{a\}$

$\text{Next}(B) = \text{Fr}(S) \cup \{c\} \cup \text{Fr}(C) \setminus \{\varepsilon\} \cup \text{Fr}(D)$

$\text{Next}(C) = \text{Next}(A) = \{a\}$



Example of application

Consider the following grammar G_1 :

$$\begin{array}{lll}
 S \rightarrow \underline{aSb} \mid cd \mid \underline{bSAe} & \text{Fr}(S) = \{a, c, b\} & \\
 A \rightarrow aAdB \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} & \text{Next}(A) = \{e, d\} \\
 B \rightarrow bb & \text{Fr}(B) = \{b\} & \text{Next}(B) = \text{Next}(A) = \{e, d\} \\
 & & \text{Next}(S) = \{\$, b\} \cup \text{Fr}(A) \setminus \{\varepsilon\} \cup \{e\} = \{\$, b, a, e\}
 \end{array}$$

Consider the following grammar G_2 :

$$\begin{array}{ll}
 S \rightarrow ABCe & \text{Fr}(S) = \text{Fr}(A) \setminus \{\varepsilon\} \cup \text{Fr}(B) \setminus \{\varepsilon\} \cup \text{Fr}(C) = \{a, b, c, d\} \\
 A \rightarrow aA \mid \varepsilon & \text{Fr}(A) = \{a, \varepsilon\} \\
 B \rightarrow bB \mid cB \mid \varepsilon & \text{Fr}(B) = \{b, c, \varepsilon\} \\
 C \rightarrow de \mid da \mid dA & \text{Fr}(C) = \{d\} \\
 & \text{Next}(S) = \{\$ \} \\
 & \text{Next}(A) = \text{Fr}(B) \setminus \{\varepsilon\} \cup \text{Fr}(C) \cup \text{Next}(A) \cup \text{Next}(C) = \{b, c, d, e\} \\
 & \text{Next}(B) = \text{Fr}(C) \cup \text{Next}(B) = \{d\} \\
 & \text{Next}(C) = \{e\}
 \end{array}$$

Example of application

Consider the following grammar G :

$$\begin{cases} E \rightarrow TE' \\ E' \rightarrow + \underline{TE'} \mid \varepsilon \\ T \rightarrow \underline{FT'} \\ T' \rightarrow * \underline{FT'} \mid \varepsilon \\ F \rightarrow \underline{(E)} \mid id \end{cases}$$

$$Next(E) = \{ \$,) \}$$

$$Next(E') = Next(E) \cup Next(E') = \{ \$,) \}$$

$$Next(T) = Fr(E') \setminus \{\varepsilon\} \cup Next(E) \cup Next(E') = \{ + , \$,) \}$$

$$Next(T') = Next(T) \cup Next(T') = \{ + , \$,) \}$$

$$Next(F) = Fr(T') \setminus \{\varepsilon\} \cup Next(T) \cup Next(T') = \{ * , + , \$,) \}$$

$$\mathbf{Fr(E)} = \mathbf{Fr(T)} = Fr(F) = \{ (, id \}$$

$$Fr(E') = \{ + , \varepsilon \}$$

$$\mathbf{Fr(T)} = \mathbf{Fr(F)} = \{ (, id \}$$

$$Fr(T') = \{ * , \varepsilon \}$$

$$Fr(F) = \{ (, id \}$$

Analysis table

Let G be the following grammar :

$$\begin{cases} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \varepsilon \\ F \rightarrow (E) \mid id \end{cases}$$

$$Next(E) = \{ \$,) \}$$

$$Next(E') = Next(E) \cup Next(E') = \{ \$,) \}$$

$$Next(T) = Fr(E') \setminus \{ \varepsilon \} \cup Next(E) \cup Next(E') = \{ + , \$,) \}$$

$$Next(T') = Next(T) \cup Next(T') = \{ + , \$,) \}$$

$$Next(F) = Fr(T') \setminus \{ \varepsilon \} \cup Next(T) \cup Next(T') = \{ * , + , \$,) \}$$

$$Fr(E) = Fr(T) = Fr(F) = \{ (, id \}$$

$$Fr(E') = \{ +, \varepsilon \}$$

$$Fr(T) = Fr(F) = \{ (, id \}$$

$$Fr(T') = \{ *, \varepsilon \}$$

$$Fr(F) = \{ (, id \}$$

	+	*	(	)	id	\$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow + TE'$	Error	Error	$E' \rightarrow \varepsilon$	Error	$E' \rightarrow \varepsilon$
T	Error	Error	$T \rightarrow FT'$	Error	$T \rightarrow FT'$	Error
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow * FT'$	Erreur	$T' \rightarrow \varepsilon$	Error	$T' \rightarrow \varepsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	Error

Analysis table

$$T' \rightarrow \varepsilon ?$$

Consider $W = id + id ?$ $\mathbf{E} \rightarrow \mathbf{TE'} \rightarrow \mathbf{FT'E'} \rightarrow id\mathbf{T'E'} \rightarrow idE' \rightarrow id + TE'$

Let the following grammar G :

$$\begin{cases} E \rightarrow TE' \\ E' \rightarrow + TE' \mid \varepsilon \\ T \rightarrow FT' \\ T' \rightarrow * FT' \mid \varepsilon \\ F \rightarrow (E) \mid id \end{cases}$$

$$Next(E) = \{ \$,) \}$$

$$Next(E') = Next(E) \cup Next(E') = \{ \$,) \}$$

$$Next(T) = Fr(E') \setminus \{ \varepsilon \} \cup Next(E) \cup Next(E') = \{ + , \$,) \}$$

$$\underline{Next(T') = Next(T) \cup Next(T') = \{ + , \$,) \}}$$

$$Next(F) = Fr(T') \setminus \{ \varepsilon \} \cup Next(T) \cup Next(T') = \{ * , + , \$,) \}$$

$$\mathbf{Fr(E) = Fr(T) = Fr(F) = \{ (, id \}}$$

$$Fr(E') = \{ + , \varepsilon \}$$

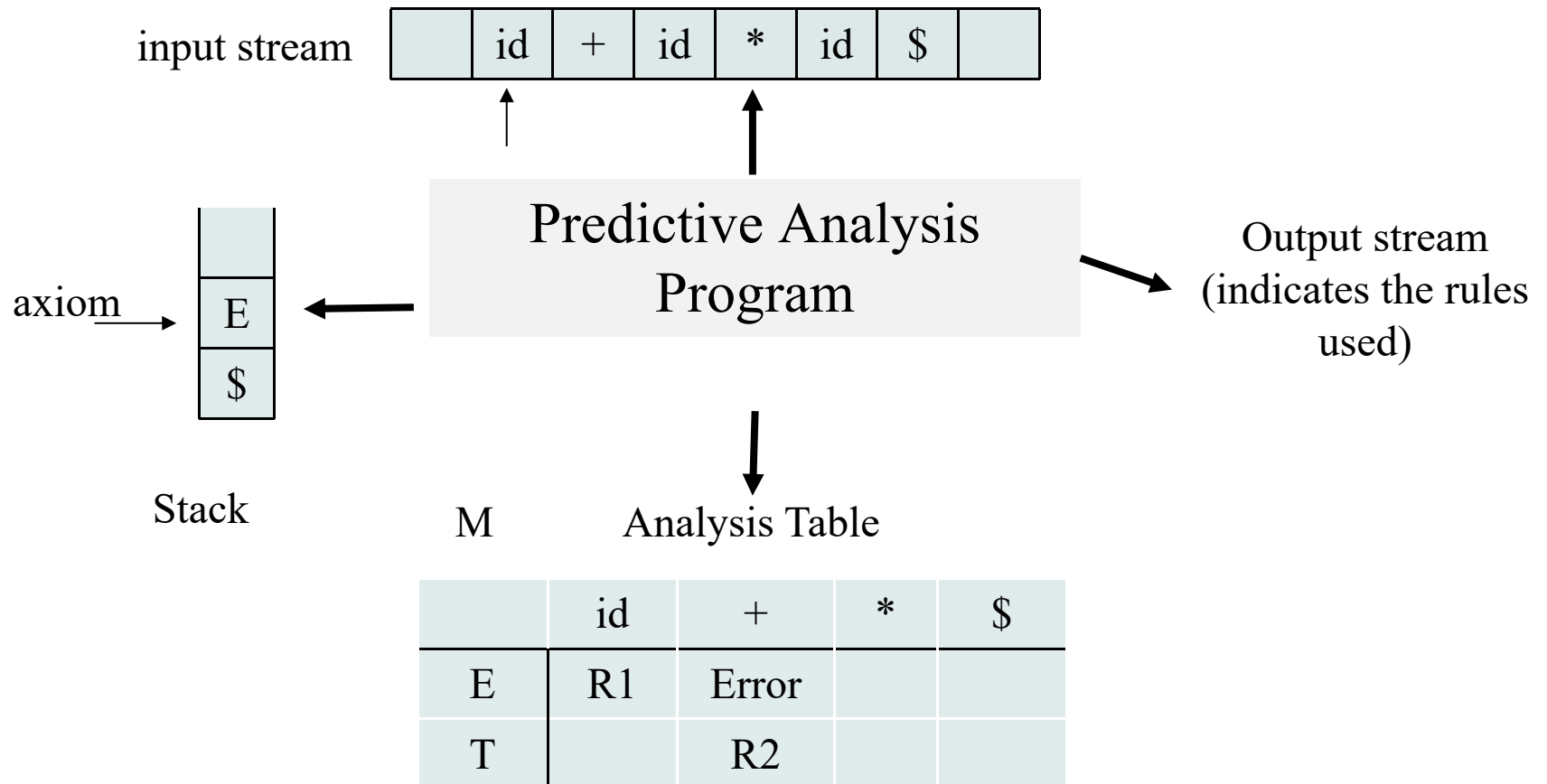
$$\mathbf{Fr(T) = Fr(F) = \{ (, id \}}$$

$$\underline{Fr(T') = \{ * , \varepsilon \}}$$

$$Fr(F) = \{ (, id \}$$

	$+$	$*$	$($	$)$	id	$\$$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow + TE'$	Error	Error	$E' \rightarrow \varepsilon$	Error	$E' \rightarrow \varepsilon$
T	Error	Error	$T \rightarrow FT'$	Error	$T \rightarrow FT'$	Error
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow * FT'$	Error	$T' \rightarrow \varepsilon$	Erreur	$T' \rightarrow \varepsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	Error

Predictive analysis



Top-down syntax analysis

Non-recursive predictive analysis

- Two-dimensional analysis table $M[A, a]$, A is non-terminal, a is terminal or $\$$.
- Initially, the stack contains the grammar axiom above the $\$$ symbol.
- X is the symbol at the top of the stack and a is the current input symbol, which determines the action of the parser. Three possibilities:
 - If $X = a = \$$, the scanner stops and the scan ends successfully.
 - If $X = a \neq \$$, the scanner removes X from the stack and advances its pointer to the next symbol.
 - If X is a non-terminal, the program consults the $M[X, a]$ entry in the M parsing table, which is either X -production or an error.
 - If X -production, replace X at the top of the stack with X -production (*).
 - If error, call the error recovery procedure.

(*) $M[X, a] = \{X \rightarrow UVW\}$, replace X at the top of the stack by WVU (with U at the top).

Top-down syntax analysis

Non-recursive predictive analysis algorithm

Input : A string w and an analysis table M for a grammar G

Output: If w is accepted by $L(G)$, a left derivation of w is *generated*, otherwise error.

Point the source pointer ps to the first symbol of $w\$$;

```
WHILE  $X \neq \$$  DO
    Consider  $X$  the symbol on the top of the stack and  $a$  the symbol pointed by  $ps$ :
    IF  $X$  is a terminal THEN
        IF  $X = a$  THEN
            remove  $X$  from the stack and move  $ps$  forward;
        ELSE
            error ();
        END IF
    ELSE /*  $X$  is a non-terminal */
        IF  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  THEN
            push  $Y_k, Y_{k-1}, \dots, Y_1$  in the stack knowing that  $Y_1$  is in the top;
            output the production  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
        ELSE
            error ();
        END IF
    END IF
END WHILE
```

	$+$	$*$	$($	$)$	id	$\$$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow + TE'$	Error	Error	$E' \rightarrow \varepsilon$	Error	$E' \rightarrow \varepsilon$
T	Erreur	Error	$T \rightarrow FT'$	Error	$T \rightarrow FT'$	Error
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow * FT'$	Error	$T' \rightarrow \varepsilon$	Error	$T' \rightarrow \varepsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	Error

Consider
 $W = id + (id * id)\$$?

End of chain
symbol

Stack top

Stack	Input	Output
$\$E$	$id + (id * id)\$$	$E \rightarrow TE'$
$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$
$\$E'T'id$	$id + (id * id)\$$	
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$
$\$E'$	$+(id * id)\$$	

Rq : We stack
upside down

Consider
 $W = id + (id * id) \$$?

	$+$	$*$	$($	$)$	id	$\$$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow + TE'$	Error	Error	$E' \rightarrow \varepsilon$	Error	$E' \rightarrow \varepsilon$
T	Error	Error	$T \rightarrow FT'$	Erreur	$T \rightarrow FT'$	Error
T'	$T' \rightarrow \varepsilon$	$T' \rightarrow * FT'$	Error	$T' \rightarrow \varepsilon$	Error	$T' \rightarrow \varepsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	Error

Stack	Input	Output
$\$E$	$id + (id * id) \$$	$E \rightarrow TE'$
$\$E'T$	$id + (id * id) \$$	$T \rightarrow FT'$
$\$E'T'F$	$id + (id * id) \$$	$F \rightarrow id$
$\$E'T'id$	$id + (id * id) \$$	
$\$E'T'$	$+(id * id) \$$	$T' \rightarrow \varepsilon$
$\$E'$	$+(id * id) \$$	$E' \rightarrow + TE'$
$\$E'T \pm$	$\pm(id * id) \$$	
$\$E'T$	$(id * id) \$$	$T \rightarrow FT'$
$\$E'T'F$	$(id * id) \$$	$F \rightarrow (E)$
$\$E'T')E($	$(id * id) \$$	
$\$E'T')E($	$(id * id) \$$	$E \rightarrow TE'$

Consider
 $W = id + (id * id) \$$?

	$+$	$*$	$($	$)$	id	$\$$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error

Stack	Input	Output		
$\$E$	$id + (id * id) \$$	$E \rightarrow TE'$	Error	$E' \rightarrow \varepsilon$
$\$E'T$	$id + (id * id) \$$	$T \rightarrow FT'$	Error	$T' \rightarrow \varepsilon$
$\$E'T'F$	$id + (id * id) \$$	$F \rightarrow id$	$\rightarrow id$	Error
$\$E'T'id$	$id + (id * id) \$$			
$\$E'T'$	$+(id * id) \$$	$T' \rightarrow \varepsilon$		
$\$E'$	$+(id * id) \$$	$E' \rightarrow + TE'$		
$\$E'T \pm$	$\pm(id * id) \$$			
$\$E'T$	$(id * id) \$$	$T \rightarrow FT'$		
$\$E'T'F$	$(id * id) \$$	$F \rightarrow (E)$		
$\$E'T')E($	$(id * id) \$$			
$\$E'T')E$	$id * id) \$$	$E \rightarrow TE'$		
$\$E'T')E'T$	$id * id) \$$	$T \rightarrow FT'$		
$\$E'T')E'T'F$	$id * id) \$$	$F \rightarrow id$		
$\$E'T')E'T'id$	$id * id) \$$			
$\$E'T')E'T'$	$* id) \$$	$T' \rightarrow * FT'$		

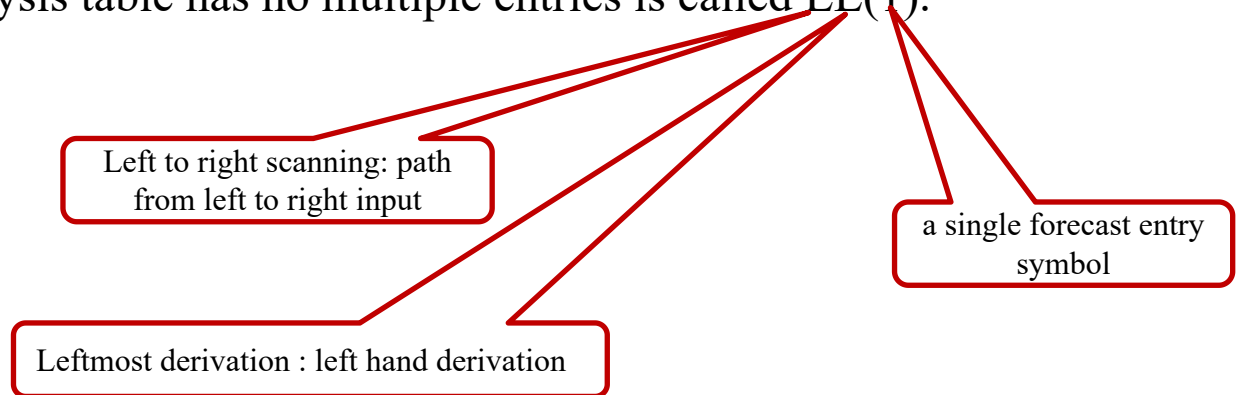
Consider
 $W = id + (id * id)$

$\$E'T$	$id + (id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$id + (id * id)\$$	$F \rightarrow id$
$\$E'T'id$	$id + (id * id)\$$	
$\$E'T'$	$+(id * id)\$$	$T' \rightarrow \varepsilon$
$\$E'$	$+(id * id)\$$	$E' \rightarrow + TE'$
$\$E'T \pm$	$\pm(id * id)\$$	
$\$E'T$	$(id * id)\$$	$T \rightarrow FT'$
$\$E'T'F$	$(id * id)\$$	$F \rightarrow (E)$
$\$E'T')E($	$(id * id)\$$	
$\$E'T')E$	$id * id)\$$	$E \rightarrow TE'$
$\$E'T')E'T$	$id * id)\$$	$T \rightarrow FT'$
$\$E'T')E'T'F$	$id * id)\$$	$F \rightarrow id$
$\$E'T')E'T'id$	$id * id)\$$	
$\$E'T')E'T'$	$* id)\$$	$T' \rightarrow * FT'$
$\$E'T')E'T'F *$	$* id)\$$	
$\$E'T')E'T'F$	$id)\$$	$F \rightarrow id$
$\$E'T')E'T'id$	$id)\$$	
$\$E'T')E'T'$	$)\$$	$T' \rightarrow \varepsilon$
$\$E'T')E'$	$)\$$	$E' \rightarrow \varepsilon$

d	$\$$
TE'	<i>Erreur</i>
<i>eur</i>	$E' \rightarrow \varepsilon$
FT'	<i>Erreur</i>
<i>eur</i>	$T' \rightarrow \varepsilon$
$\rightarrow id$	<i>Erreur</i>

LL(1) Grammar

- For some grammars, the analysis table may have multiple entries.
- A grammar whose analysis table has no multiple entries is called LL(1).



A grammar is called LL(1) ➔ It means that this grammar is :

- Left factorized
- Left non-recursive
- Unambiguous